

Integrating the Acrome Smart Motion Car with Artificial Intelligence: Challenges and Solutions

The Acrome Smart Motion Car is somewhat similar to a remote-controlled vehicle. Its working principle is based on the SMD boards inside it, which enable the motors to rotate. These motors have four operation modes, but the mode we use in this project is the Velocity mode.

The main objective of this project is as follows: to integrate the Acrome Smart Motion Kit with artificial intelligence and make it operational. For instance, there are some pre-written codes for the Smart Motion Kit, such as the "move forward" function. The AI processes the given command and sequentially calls the appropriate functions accordingly. For example, if the command is "move forward 50 cm and then turn right," the car will first move forward 50 cm, then turn right, and finally stop at the designated position. These operations were already part of the Acrome AI Project folder under the Motion Kit (Car) Control section.

However, the main issue here was that the Raspberry Pi couldn't automatically switch between Wi-Fi networks. Additionally, even if it did switch, the Raspberry Pi's IP address would change, causing the device (mobile phone) to lose communication with it.

APPLICATION STEPS:

1. To use the application, you need an Android phone. The mobile app has been specifically developed for Android using Flutter. Below are the steps to set it up:
 - a. In the repository, I've provided a link where you can directly download the application. If you prefer not to build the app yourself, simply use the link provided at the bottom of the document or in the README section to install the app.
 - b. If you want to build the app manually, download the mobile_control folder from the repository. Open the folder in VS Code (or your preferred IDE) after ensuring Flutter is installed on your system. Run the following command to build the APK: **flutter build apk --split-per-abi**
 - c. If you wish, you can modify the source code to adjust the design according to your preferences. Simply make changes to the files in the mobile_control folder before building the APK.

2. You need to have a Raspberry Pi device to run the application.
 - a. You should upload appfinal.py file to the Raspberry Pi. (I recommend using MobaXterm due to its ease of use.)
 - b. When flashing the Raspberry Pi image, make sure the Wi-Fi option is enabled. If Wi-Fi is not enabled during the setup process, you may need to manually configure the Wi-Fi settings later, which could complicate the setup.

STEPS BY STEPS



Figure 1: Raspberry pi Imager

Since this project has been tested on a Raspberry Pi 4, it is recommended to use a Raspberry Pi 4 for the best experience. Additionally, select Raspberry Pi OS (64-bit) as the operating system.

After clicking the Next button, enter the general information in the OS Customization section according to your preferences. For example:

OS Customization

GENERAL SERVICES OPTIONS

☐ Set hostname: raspberrypi .local

☒ Set username and password

Username: pi

Password: ●●●●●●●●

☒ Configure wireless LAN

SSID: Acrome_Lab

Password: ●●●●●●●●

☐ Show password ☐ Hidden SSID

Wireless LAN country: TR

☒ Set locale settings

Time zone: Europe/Istanbul

Keyboard layout: tr

SAVE

Figure 2: OS Customization.

Note: Please ensure that Configure Wireless LAN is enabled and enter your modem's SSID and password.

Then You should download this app on your phone :

<https://dosya.co/6xxy1fmu7356/app-armeabi-v7a-release.apk.html>

To find the IP address of your Raspberry Pi while it's running, follow these steps:

Install the application you downloaded.

After the installation, open the application.

The IP scanner screen will appear, displaying the available devices on your network.

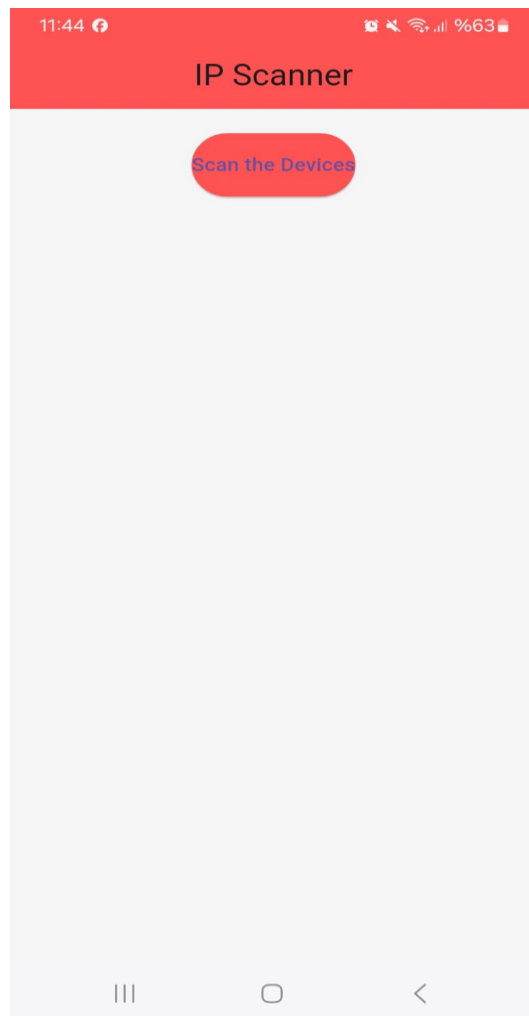


Figure3: IP Scanner

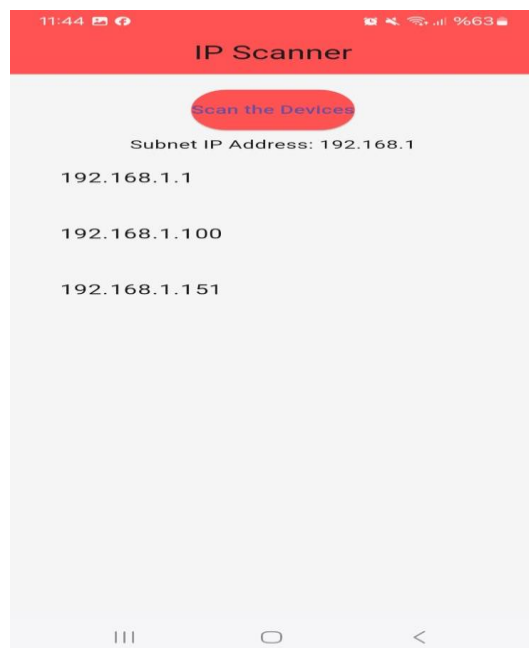


Figure4: Devices (.1 is a modem and

.100 is a router. That's why I chose 192.168.1.151 . Also this scanner just find Linux devices.)

Then, I connected to the Raspberry Pi 4 using MobaXterm.

First step: You have to upload appfinal.py and requirements.txt files. Then you must write to Bash this command:

pip install -r requirements.txt --break-system-packages

After that you should write this command:

sudo systemctl status NetworkManager

And you should see like this:

```
pi@raspberrypi:~$ sudo systemctl status NetworkManager
● NetworkManager.service - Network Manager
   Loaded: loaded (/lib/systemd/system/NetworkManager.service; enabled; preset: enabled)
   Active: active (running) since Tue 2024-11-19 16:44:29 +03; 1 month 17 days ago
     Docs: man:NetworkManager(8)
   Main PID: 558 (NetworkManager)
      Tasks: 3 (limit: 1575)
         CPU: 443ms
    CGroup: /system.slice/NetworkManager.service
            └─558 /usr/sbin/NetworkManager --no-daemon

Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2584] policy: set 'preconfigured' (wlan0) as default for IPv4 routing and DNS
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2787] device (wlan0): state change: ip-config -> ip-check (reason 'none', sys-iface-state='unmanaged')
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2843] device (wlan0): state change: ip-check -> secondaries (reason 'none', sys-iface-state='unmanaged')
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2847] device (wlan0): state change: secondaries -> activated (reason 'none', sys-iface-state='unmanaged')
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2855] manager: NetworkManager state is now CONNECTED_SITE
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2900] device (wlan0): Activation: successful, device activated.
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2909] manager: NetworkManager state is now CONNECTED_GLOBAL
Nov 19 16:44:37 raspberrypi NetworkManager[558]: <info> [1732023877.2928] manager: startup complete
Nov 19 16:44:40 raspberrypi NetworkManager[558]: <info> [1732023880.7900] dhcp6 (wlan0): activation: beginning transaction (timeout in 45 seconds)
Nov 19 16:44:42 raspberrypi NetworkManager[558]: <info> [1732023882.6663] agent-manager: agent[aaf39b0245e6629e,::1.34/org.freedesktop.nm-applet/1000]: agent[...]
```

Figure 5: Command Shell

After these steps, if you are going to use Raspberry somewhere else (on another network), write this line:

sudo nmcli dev wifi hotspot ifname wlan0 ssid "Hotspot_name" password "Password"

Like this:

```
pi@raspberrypi:~$ sudo nmcli dev wifi hotspot ifname wlan0 ssid pi password raspberry
```

Figure 6: Create Hotspot

However, since you switched from the hotspot to Wi-Fi (which is the right decision), if you later move the Raspberry Pi to another location (such as your home or another lab), I recommend re-enabling the hotspot mode. The command to do this is as follows:

sudo nmcli connection up Hotspot

```
pi@raspberrypi:~$ sudo nmcli connection up Hotspot
```

Figure 7: Open the Hotspot

Now, we've reached an important step. As soon as the Raspberry Pi starts, the appfinal.py file will run automatically. I assume you've already installed the necessary libraries using pip install at the beginning.

Please pay close attention to these steps and follow them carefully.

Firstly, you should enter the following command:

```
sudo nano /etc/systemd/system/run_script.service
```

You will see an empty file. Then, you should add the following lines to this file:

```
[Unit]
Description=Run Python script on startup
After=network.target

[Service]
ExecStart=/usr/bin/python3 /home/pi/your_script.py
WorkingDirectory=/home/pi
User=pi
Group=pi
Restart=always

[Install]
WantedBy=multi-user.target
```

Figure 8: Important Lines

Of course, you change /home/pi/your_script.py and User. I do not know what you write user name but if you upload appfinal.py file to /home/pi folder. Then you write /home/pi/appfinal.py. These lines:

```
[Unit]

Description=Run Python script on startup

After=network.target


[Service]

ExecStart=/usr/bin/python3 /home/pi/appfinal.py
```

WorkingDirectory=/home/pi

User=pi

Group=pi

Restart=always

[Install]

WantedBy=multi-user.target

After save that file, you should enter these commands:

sudo systemctl daemon-reload

sudo systemctl enable run_script.service

sudo systemctl start run_script.service

Command Like this:

```
sudo systemctl daemon-reload
sudo systemctl enable run_script.service
sudo systemctl start run_script.service
```

Figure 9: run_script.service is running

```
pi@raspberrypi:~ $ sudo systemctl status run_script.service
● run_script.service - Run Python script on startup
   Loaded: loaded (/etc/systemd/system/run_script.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-01-06 12:11:38 +03; 13s ago
     Main PID: 637 (python3)
        Tasks: 3 (limit: 1575)
            CPU: 3.418s
    CGroup: /system.slice/run_script.service
            └─637 /usr/bin/python3 /home/pi/appfinal.py
              └─853 /usr/bin/python3 /home/pi/appfinal.py

Jan 06 12:11:38 raspberrypi systemd[1]: Started run_script.service - Run Python script on startup.
Jan 06 12:11:44 raspberrypi python3[637]: * Serving Flask app 'appfinal'
Jan 06 12:11:44 raspberrypi python3[637]: * Debug mode: on
```

Figure 10: Check the run_script.service

If you followed the steps correctly, you should see a screen like the one in Figure 10.

The Raspberry Pi part is now complete, and there is a video showing the progress on the mobile side.

<https://drive.google.com/file/d/1HpLvHrnCPlmwpBCMEQtaDkjlUCLsbwnX/view?usp=sharing>

In this video, I'll briefly explain the steps I followed:

Scanning for Devices

First, I scanned for devices and connected to the Raspberry Pi's hotspot to establish the connection (IP: 10.42.0.1).

Connecting to Wi-Fi

After that, I clicked Next and entered the name (SSID) and password of the Wi-Fi I wanted to use. This instructed the Raspberry Pi to disable the hotspot and connect to the specified Wi-Fi network.

If the connection fails, don't worry. The hotspot will reopen automatically, but the process might take a bit longer.

Switching to the Same Network

Once the Raspberry Pi connected to the Wi-Fi, I changed my phone's Wi-Fi settings to join the same network as the Raspberry Pi. I ensured both devices were on the same network.

Finding the New IP Address

Next, I scanned for the Raspberry Pi's new IP address, as the others listed were for the modem or router.

API Key Screen

The application then displayed an API key screen. This was for the Groq Cloud service we used for artificial intelligence. I skipped this step since I had already entered the API key previously.

Executing the Command

Finally, I entered a command for the AI to execute. The AI successfully processed the command and performed the requested task.

Note: It also supports voice input, and the screen looked like this:

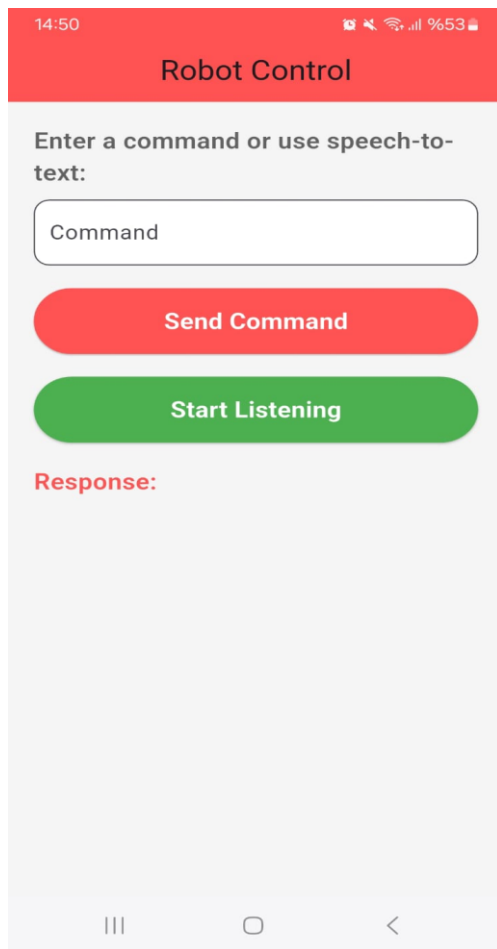


Figure 11: New Version